

알고리즘으로 보는 pandas와 Datathon

정렬 · 힙 · 탐색 · 그래프 알고리즘은 데이터 분석의 어디에 숨어 있나

[Alg 13 Heap sort] = Algorithms 강의 슬라이드 번호, [통계책 3장] = “pandas 흐름으로 따라가는 통계” 문서의 장.
시간은 이 컴퓨터(Apple Silicon, numpy 2.1, pandas 2.2)에서 3회 중 최솟값으로 직접 측정했습니다.

결론부터: 쓸 일이 있나?

직접 구현할 일은 거의 없지만, **pandas·numpy·sklearn** 내부에서 이미 쓰이고 있습니다. 알고 있으면 (1) 왜 어떤 코드가 수백 배 느린지 이해하고, (2) 함수의 옵션(kind=, max_leaf_nodes=)을 의미 있게 고르고, (3) 결과(트리 모형, 분위수)를 정확히 해석할 수 있습니다.

배운 것	pandas/라이브러리에서 쓰이는 곳	장	관련도
Big-O, RAM 모델	반복문 vs 벡터화, iterrows 금지	1	★★★★
Quick sort + Heap sort	sort_values 기본값 = introsort (둘의 조합)	2	★★★★
Merge sort, 안정성	kind="stable" (timsort), 다중 키 정렬	2	★★★★
Counting / Radix sort	groupby 내부의 counting sort, 정수 정렬	2, 4	★★★☆
정렬 하한 $\Omega(n \log n)$	왜 radix가 더 빠를 수 있나	2	★☆☆
Selection (quickselect)	median, quantile, nlargest	3	★★★★
Heap	top-k, 결정트리 best-first, Dijkstra	3, 6, 9	★★★☆
Binary search	searchsorted, pd.cut, merge_asof	5	★★★★
Hashing	groupby, merge, value_counts	4	★★★★
DFS / BFS, 트리	결정트리 학습·출력, 연결요소(중복 제거)	6	★★★☆
Heuristic, greedy	결정트리 분할, 변수 선택 (stepwise)	7	★★★☆
Minimax	직접은 없음. 개념: minimax 추정, minimax 공정성	8	★☆☆
Dijkstra	이번 datathon엔 없음. 네트워크·교통 데이터용	9	☆☆☆
Divide & conquer, 점화식	split-apply-combine, bootstrap 비용	10	★★☆☆

※ 4시간짜리 채용 bias datathon에서 실제로 판단에 영향을 주는 것은 1-7장입니다. 8-10장은 “어디에 쓰이는지 궁금할 때” 읽으면 됩니다.

먼저 읽기: 분석 전체 흐름과 책 안내

네 권의 역할

표시	책 (파일)	역할
[흐름]	pandas 흐름으로 따라가는 통계 (pandas-stats-walkthrough)	기초: 평균·SD·정규·상관·회귀·검정을 pandas 순서대로
[응용]	채용 데이터로 다시 배우는 통계 (hiring-statistics-book)	응용: 통계 회귀, logistic, Bayesian, 공정성으로 결론 내기
[알고]	알고리즘으로 보는 pandas (algorithms-for-datathon)	도구: 빠르고 정확한 코드, 결정트리·greedy 보조 분석
[실습]	numpy·pandas로 직접 만드는 통계 (stats-basics-workbook)	실습: 빈칸 노트북으로 통계 공식을 직접 구현, 필요한 곳에 알고리즘 적용

※ 아래 표의 작은 색 주석이 “이 단계는 어느 책 몇 장”입니다. 예: [흐름 5] = 흐름 책 5장, [응용 6.9] = 응용 책 6.9절, [알고 4] = 알고리즘 책 4장. 0장 = 응용 책 앞의 “데이터 미리보기와 pandas 작동 원리”.

Datathon 분석 흐름: 데이터를 받은 순간부터 발표까지

단계	하는 일	대표 pandas 코드	해당 장
①	질문을 통계 질문으로	–	[응용 1] [흐름 1 추정·가설·검증 상자]
②	불러오고 구조 확인	read_csv, head, info	[응용 0.1–0.3] [흐름 1]
③	변수 분류 (결과 Y / 보호속성 A / 자격 X)	dtypes, value_counts	[응용 2] [흐름 1.1]
④	결측·이탈값 처리	isna, dropna, quantile	[응용 0.7, 2.4] [흐름 1.3, 2.3]
⑤	분포 요약 (중심, 퍼짐, 정규성)	describe, std, $(x-x.mean())/x.std()$	[흐름 2, 3] [응용 3.1] [알고 3 분위수] [실습 1–3]
⑥	그룹별 합계들과 신뢰구간	groupby().mean()	[응용 0.5, 3.2] [흐름 4] [알고 4 groupby 내부] [실습 4]
⑦	교란 변수 확인 (Simpson)	crosstab, groupby([a,b])	[응용 3.3] [흐름 12 ⑧]
⑧	가설검정 (차이가 우연인가)	proportions_ztest, chi2_contingency, ttest_ind	[흐름 5, 11, 12] [응용 5] [실습 5–7]
⑨	상관과 단순회귀	corr, pd.cut, smf.ols	[흐름 6–8] [알고 5 pd.cut] [실습 8–9]
⑩	통제 회귀와 interaction (핵심)	smf.ols("y ~ A + X"), A*B	[응용 0.8–0.9, 6] [흐름 13] [실습 10]
⑪	다른 방법으로 재확인	smf.logit, rng.beta, DecisionTreeClassifier	[응용 7, 8] [흐름 9 bootstrap] [실습 6, 11]
⑫	공정성 지표와 한계	rate["F"]/rate["M"], idxmin	[응용 9] [알고 8 minimax]
⑬	정리와 발표	–	[응용 9.2 결론 문장, 11 체크리스트] [흐름 14 요약] [알고 11]

※ 이론이 막히면: 추정량·MoM·MLE [흐름 9–10] [응용 4], 가설 세우기·검정력 [흐름 11], 코드가 느리면 [알고 1–2, 4]. 한 번에 돌려 보기: python3 algo_toolkit.py hiring_practice.csv (⑥⑦⑨⑪⑫ 요약).

6시간 읽기 계획 (읽기 편한 순서)

시간	무엇을	어디를	분
0:00	데이터와 pandas 감 잡기	[응용 0] 전체, [응용 1] 전체 지도	30
0:30	기초 통계 1: 요약과 분포	[흐름 1-3] (<code>describe</code> , 표준화, 정규근사)	50
1:20	기초 통계 2: 추정과 검정	[흐름 4-5] (신뢰구간, $z/\chi^2/t$)	40
2:00	휴식		10
2:10	기초 통계 3: 상관과 회귀	[흐름 6-8] (회귀 효과, RMSE, 잔차도)	50
3:00	검정 고르는 법	[흐름 12] 검정 도구상자 (표 + 관심 있는 카드만)	30
3:30	응용 1: 통제와 interaction	[응용 3, 6] + 각 장 끝 “pandas 코드” 직접 실행	50
4:20	휴식		10
4:30	응용 2: 재확인	[응용 7] logistic, [응용 8] Bayesian (8.4 “왜”는 꼭)	40
5:10	결론과 한계	[응용 9], [응용 11] 체크리스트, [알고 표지 결론표]	20
5:30	도구 실행과 발표 문장 연습	<code>algo_toolkit.py</code> 실행, [응용 9.2] 문장을 내 말로	30
(다음 날)	코드로 손에 익히기	[실습 0-12] 빈칸 노트북 (<code>stats_basics_practice.ipynb</code>)	120

※ 범위 밖 (학부 1-2학년 목표를 넘는 것): `sklearn`을 쓰는 [알고 6-7]의 결정트리와 `advanced/` 폴더의 분류 워크북은 나중에 선택으로.

※ 건너뛰어도 되는 곳 (나중에 참고용): [흐름 9-11, 13], [응용 4, 10], [알고 1-10] 본문, 모든 연습 문제. 시간이 남으면 [흐름 15] 연습 문제 1-8번부터.

※ 각 장은 새 페이지에서 시작하고, 목차의 항목을 누르면 그 쪽으로 이동합니다.

목차 (Contents)

먼저 읽기: 분석 전체 흐름과 책 안내	2
1 Big-O와 RAM 모델: 왜 반복문을 쓰지 말라고 하나	5
1.1 같은 $\Theta(n)$ 인데 230배 차이	5
1.2 pandas에서의 실제 예	5
2 정렬: sort_values 안에서 일어나는 일	6
2.1 kind= 옵션은 강의 목차 그 자체	6
2.2 측정: 100만 개	6
2.3 Stability가 실제로 필요한 순간	7
2.4 통계에서 정렬이 쓰이는 곳	7
3 선택(selection)과 힙: median, quantile, top-k	8
3.1 중앙값을 구하려고 전부 정렬할 필요는 없다	8
3.2 Top-k: nlargest	8
4 해싱과 counting sort: groupby와 merge	9
4.1 groupby 내부	9
4.2 merge: $O(nm)$ vs $O(n + m)$	9
5 이진 탐색: searchsorted와 pd.cut	10
5.1 정렬되어 있으면 $O(\log n)$	10
5.2 어디에 숨어 있나	10
6 트리, DFS, BFS: 결정트리와 연결요소	11
6.1 결정트리 = 데이터로 만든 이진트리	11
6.2 트리를 만들고 읽는 순서 = DFS / best-first	11
6.3 BFS/DFS가 직접 필요한 경우: 연결요소	11
7 Heuristic과 greedy: 최적해 대신 좋은 해	13
7.1 결정트리는 greedy heuristic	13
7.2 변수 선택: forward stepwise (greedy)	13
7.3 Heuristic function과의 관계	13
8 Minimax: 직접은 없지만 같은 생각이 있다	14
8.1 원래 의미	14
8.2 같은 원리가 나오는 곳	14
9 Dijkstra: 이번 datathon에는 필요 없다	15
9.1 언제 쓰나	15
9.2 알고리즘의 연결고리	15

10 분할정복과 점화식: 분석 파이프라인의 비용	16
10.1 Split-apply-combine	16
10.2 재표본 방법의 비용	16
10.3 강의의 다른 알고리즘이 쓰이는 곳	16
11 정리: datathon 실전 규칙	17
12 연습 문제	18
13 풀이	19

1 Big-O와 RAM 모델: 왜 반복문을 쓰지 말라고 하나

[Alg 04 Big-O, 06 RAM model, 03 Why constants matter less]

1.1 같은 $\Theta(n)$ 인데 230배 차이

```
x = np.random.normal(size=1_000_000)
# (a) 파이썬 반복문                                0.0328초
t = 0.0
for v in x: t += v
# (b) 벡터화                                          0.00014초
x.mean()
```

두 방법 모두 원소를 한 번씩 보는 $\Theta(n)$ 입니다. 그런데 약 **230배** 차이가 납니다.

강의의 “상수는 덜 중요하다”와 충돌하나?

- 강의: n 이 커지면 $\Theta(n)$ 과 $\Theta(n^2)$ 의 차이가 상수 배 차이를 압도한다. 여전히 맞습니다 (아래 4장의 1400배).
- 하지만 같은 차수끼리는 상수가 전부입니다. RAM 모델은 “기본 연산 1회 = 상수 시간”을 가정하는데, 파이썬에서는 `t += v` 한 번이 타입 확인, 객체 생성, 참조 카운트 등 수십 개의 기계 명령입니다. numpy는 같은 루프를 C로, 연속 메모리 위에서, SIMD로 돌립니다.

※ 그래서 pandas의 제1원칙: 행 단위 반복을 열 단위 연산으로 바꿔라 (vectorization).

1.2 pandas에서의 실제 예

test_score의 z-score (582행)	시간	배율
<code>for _, r in d.iterrows(): ...</code>	5.1 ms	110배
<code>(d.test_score - d.test_score.mean()) / d.test_score.std()</code>	0.046 ms	1

※ `iterrows`는 행마다 Series 객체를 새로 만듭니다. `apply(axis=1)`도 비슷하게 느립니다. 600행에서는 체감이 없지만, 수십만 행 datathon 데이터에서는 몇 초와 몇 분의 차이가 됩니다.

Datathon에서는

- 열 연산 (+ - * /, `np.where`, `.str`, `.dt`), `groupby`, `merge`를 우선 쓰고, 반복문은 마지막 수단으로.
- 코드가 느리면 먼저 “이 루프의 차수가 n^2 인가?”(4장)를 확인하고, 그다음 “파이썬 루프인가?”를 확인합니다.

2 정렬: sort_values 안에서 일어나는 일

[Alg 10 Merge sort, 13 Heap sort, 14–15 Quick sort, 16 Lower bound, 17 Counting/Radix]

2.1 kind= 옵션은 강의 목차 그 자체

```
df.sort_values("test_score", kind="quicksort") # 기본값
# kind: {"quicksort", "mergesort", "heapsort", "stable"}
```

kind	실제 알고리즘 (numpy 문서)	최악	stable
quicksort	introsort : quick sort, 재귀가 깊어지면 heap sort로 전환	$O(n \log n)$	✗
heapsort	heap sort	$O(n \log n)$	✗
mergesort, stable	실수: timsort (merge + insertion), 작은 정수: radix sort	$O(n \log n)$	✓

※ Introsort = 강의 14–15의 quick sort (평균 $\Theta(n \log n)$, 최악 $\Theta(n^2)$) + 강의 13의 heap sort (최악 $\Theta(n \log n)$). Quick sort가 나쁜 pivot을 계속 고르면 heap sort로 바뀌서, 평균은 *quick sort*만큼 빠르고 최악은 *heap sort*만큼 안전합니다. 두 강의를 합친 실전형입니다.

2.2 측정: 100만 개

데이터	방법	시간
무작위 실수	quicksort (introsort)	21 ms
	mergesort / stable (timsort)	68 / 70 ms
	heapsort	74 ms
이미 정렬된 실수	quicksort	20 ms
	stable (timsort)	0.9 ms
0–99 정수 (int8)	quicksort	19 ms
	stable (radix sort)	0.7 ms

표에서 배우는 것

1. 무작위 자료에서는 **quicksort**가 가장 빠름. 셋 다 $\Theta(n \log n)$ 이지만 quick sort는 메모리를 순서대로 읽어서 (cache-friendly) 상수가 작습니다. 그래서 기본값입니다.
2. **Heap sort**가 가장 느림. 최악 보장은 좋지만, 힙에서 부모–자식 사이를 건너뛰며 접근해서 캐시 효율이 나쁩니다. 그래서 단독보다는 introsort의 안전장치로 쓰입니다.
3. 정렬된 자료에서 **timsort**가 **23배** 빠름. 이미 정렬된 구간(run)을 찾아 합치므로 거의 $\Theta(n)$. Insertion sort의 best case $\Theta(n)$ [Alg 9]과 같은 원리입니다. 시계열처럼 “거의 정렬된” 자료에 유리합니다.
4. 작은 정수에서 **radix sort**가 **27배** 빠름. 비교 정렬의 하한 $\Omega(n \log n)$ [Alg 16]은 비교만 쓰는 정렬에 대한 것입니다. Counting/radix sort는 값을 직접 인덱스로 쓰므로 $\Theta(n + k)$, $\Theta(d(n + k))$ 로 그 하한을 피합니다 [Alg 17].

2.3 Stability가 실제로 필요한 순간

Stable sort: 키가 같은 원소들의 원래 순서를 유지합니다. 여러 키로 정렬할 때 중요합니다.

```
# 부서별로, 같은 부서 안에서는 점수 내림차순
d.sort_values("test_score", ascending=False)\
.sort_values("department", kind="stable")          # 2단계 정렬: 두 번째는 반드시 stable
d.sort_values(["department", "test_score"], ascending=[True, False]) # 같은 결과, 권장
```

※ 두 번째 정렬이 unstable이면 같은 부서 안의 점수 순서가 깨질 수 있습니다. Radix sort가 낮은 자리부터 stable하게 정렬하는 원리 [Alg 17]와 정확히 같습니다: 덜 중요한 키 먼저, 더 중요한 키를 나중에 stable하게.

2.4 통계에서 정렬이 쓰이는 곳

- **Order statistics** $x_{(1)} \leq \dots \leq x_{(n)}$ = 정렬 결과 그 자체 [통계책 2장] (강의노트 Def 2.1).
- **Empirical CDF, Q-Q plot**: 정렬된 자료를 이론 분위수와 짝지음 [통계책 3장].
- **Rank**: `s.rank()` = 정렬 후 위치. Mann–Whitney, Spearman 상관 같은 비모수 검정의 재료.
- `drop_duplicates`, `merge_asof`, 시계열 리샘플링은 정렬된 상태를 가정하거나 내부에서 정렬합니다.

3 선택(selection)과 힙: median, quantile, top- k

[Alg 13 Heap sort, 14–15 Quick sort (partition)]

3.1 중앙값을 구하려고 전부 정렬할 필요는 없다

k 번째 작은 값만 필요하면 **quickselect**를 씁니다. Quick sort의 Partition을 한 번 하고, k 가 있는 한쪽만 재귀합니다.

$$\begin{aligned} \text{Quick sort: } T(n) &= 2T(n/2) + \Theta(n) = \Theta(n \log n), \\ \text{Quickselect: } T(n) &= T(n/2) + \Theta(n) = \Theta(n) \text{ (평균)}. \end{aligned}$$

※ 두 번째 점화식은 $n + n/2 + n/4 + \dots \leq 2n$. 강의 18–19의 recursion tree로 바로 풀립니다.

100만 개의 중앙값	시간
<code>np.sort(x)[n//2]</code> (전부 정렬)	21 ms
<code>np.partition(x, n//2)[n//2]</code> (introselect)	7.3 ms
<code>np.median(x)</code> (내부에서 partition 사용)	9.0 ms

※ `np.quantile`, `Series.quantile`, `describe()`의 25/50/75%도 내부에서 `partition`을 씁니다 (소스에서 확인). 즉 통계 책 2장의 사분위수 계산이 `quickselect`입니다.

3.2 Top- k : `nlargest`

100만 개 중 상위 10개	방법	시간
<code>s.sort_values(ascending=False).head(10)</code>	전부 정렬 $\Theta(n \log n)$	77 ms
<code>heapq.nlargest(10, x)</code>	크기 k 힙 $\Theta(n \log k)$ (파이썬)	24 ms
<code>s.nlargest(10)</code>	selection + 10개만 정렬	10 ms

- **힙 방법**: 크기 k 인 min-heap을 유지. 새 원소가 힙의 최솟값(루트)보다 크면 교체하고 `HeapifyDown` $O(\log k)$. 강의 13의 힙 연산 그대로입니다.
- **pandas 방법** (소스 확인): `kth_smallest(quickselect 계열)`로 k 번째 값을 찾고, 그보다 큰 원소만 골라 `merge sort`.

※ 힙은 “전체 정렬은 필요 없고 가장 큰/작은 것만 반복해서 꺼내고 싶을 때”의 자료구조입니다. 이 성질 때문에 6장 결정트리 (best-first)와 9장 Dijkstra에서 priority queue로 다시 나옵니다.

Datathon에서는

- “점수 상위 10명”, “합격률 낮은 부서 3개”는 `nlargest/nsmallest`.
- 분위수, 중앙값, IQR은 정렬 없이 빠르게 계산되므로 큰 데이터에서도 `describe()`를 부담 없이 씁니다.

4 해싱과 counting sort: groupby와 merge

[Alg 17 Counting sort]

4.1 groupby 내부

`df.groupby("gender").hired.mean()`은 대략 세 단계입니다.

1. **Factorize** (해싱): "F", "M" → 정수 코드 0, 1. 해시 테이블로 평균 $O(n)$.
2. **Counting sort**: 코드별로 행을 모음. pandas 소스의 주석 그대로 “`groupsort_indexer implements counting sort`”. 그룹 수 k 가 작으므로 $\Theta(n + k)$.
3. 그룹별로 합과 개수를 누적해 평균 계산.

※ 강의 17의 counting sort: (1) 각 키의 개수를 세고 (2) 누적합으로 시작 위치를 정하고 (3) 원소를 제자리에 놓는다. `groupby`가 정확히 이렇게 그룹의 시작/끝 위치를 만듭니다. 그래서 600행 `groupby`가 0.09 ms.

4.2 merge: $O(nm)$ vs $O(n + m)$

3000행 × 3000행 조인	복잡도	시간
이중 for문 (모든 쌍 비교)	$\Theta(nm)$	448 ms
<code>a.merge(b, on="k")</code> (hash join)	$\Theta(n + m)$ 평균	0.32 ms

1400배. 1장과 달리 이번에는 차수의 차이입니다. 행이 10배 늘면 이중 루프는 100배, hash join은 10배 느려집니다.

※ Hash join: 작은 쪽 키로 해시 테이블을 만들고 ($O(m)$), 다른 쪽을 훑으며 조회 ($O(n)$, 조회당 평균 $O(1)$).

Datathon에서는

여러 테이블(지원자, 면접, 부서 정보)을 받으면 반드시 `merge`로 합칩니다. 합친 뒤 `len()`이 예상과 같은지 확인하세요. 키가 중복되면 행이 곱으로 늘어납니다 (`validate="one_to_one"` 옵션).

5 이진 탐색: `searchsorted`와 `pd.cut`

[Alg 07 Search, Case study 1.1 Binary search]

5.1 정렬되어 있으면 $O(\log n)$

정렬된 100만 개에서 1000개 값의 위치 찾기	복잡도	시간
선형 탐색 (<code>np.argmax(xs >= v)</code>)	$\Theta(n)$ 각각	79 ms
<code>np.searchsorted(xs, q)</code>	$\Theta(\log n)$ 각각	0.19 ms

※ 강의 case study의 전제조건 “배열 A 가 정렬되어 있음”이 그대로 적용됩니다. `searchsorted`에 정렬되지 않은 배열을 넣으면 오류 없이 틀린 답이 나옵니다. 정확성 증명의 전제조건이 실전에서 왜 중요한지 보여 주는 예입니다.

5.2 어디에 숨어 있나

- `pd.cut(score, bins)`: 각 점수가 어느 구간인지 = 경계값 배열에서 이진 탐색 (소스에서 `searchsorted` 확인). 통계책 7장 “평균의 그래프”에서 쓴 바로 그 함수입니다 [통계책 7장].
- `pd.merge_asof`: “가장 가까운 이전 시점”과 조인. 양쪽이 정렬되어 있어야 함.
- 정렬된 인덱스에서 `df.loc["2024-01":"2024-03"]` 같은 범위 선택.
- `np.quantile`의 역함수 격인 “이 점수는 몇 백분위?” = `np.searchsorted(np.sort(s), 85) / len(s)` (통계책 3장의 95.7%).

6 트리, DFS, BFS: 결정트리와 연결요소

[Alg 12 Trees, 13 Heap (완전이진트리)]

6.1 결정트리 = 데이터로 만든 이진트리

```
from sklearn.tree import DecisionTreeClassifier, export_text
X = pd.get_dummies(d[["gender", "department", "degree", "test_score", "years_experience"]],
drop_first=True).astype(float)
clf = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X, d.hired)
print(export_text(clf, feature_names=list(X.columns)))
```

```
|--- department_Marketing <= 0.50                                     (Engineering)
| |--- test_score <= 68.50
| | |--- years_experience <= 6.50   -> class 0
| | |--- years_experience > 6.50   -> class 1
| |--- test_score > 68.50
| | |--- gender_M <= 0.50           -> class 0                                     (여성)
| | |--- gender_M > 0.50           -> class 1                                     (남성)
|--- department_Marketing > 0.50                                     (Marketing)
| |--- test_score <= 79.50 ... -> class 0
```

해석: 트리가 스스로 “Engineering + 점수 68.5 초과” 그룹에서 성별로 분할했습니다. 점수가 높은 Engineering 지원자 중 남성은 합격, 여성은 불합격으로 예측됩니다. 통계책·이전 책에서 회귀 interaction으로 찾은 “Engineering에서의 성별 격차”를 전혀 다른 방법이 다시 발견한 것입니다.

※ Feature importance: Marketing 0.44, 점수 0.34, 경력 0.18, 성별 0.05, 학위 0. 다만 트리의 importance는 효과의 크기나 유의성이 아니므로 결론은 회귀로 내리고, 트리는 “패턴 탐색” 용으로 씁니다.

6.2 트리를 만들고 읽는 순서 = DFS / best-first

sklearn 소스(_tree.pyx)에는 두 가지 builder가 있습니다.

builder	언제	자료구조 = 탐색 방식
DepthFirstTreeBuilder	기본 (max_depth 등)	스택 = DFS
BestFirstTreeBuilder	max_leaf_nodes 지 정 시	우선순위 큐 (힙) = 불순도 감소가 가장 큰 노드부터

- export_text의 출력 순서는 전위 순회 (preorder DFS): 노드 → 왼쪽 서브트리 전체 → 오른쪽 서브트리.
- max_leaf_nodes=4로 바꾸면 힙에서 “가장 이득이 큰 잎”부터 꺼내 분할합니다. 결과: Marketing → 0; Engineering은 점수 68.5로 나누고, 그 아래 낮은 점수 쪽만 경력으로 한 번 더 분할. 잎이 4 개뿐이라 성별 분할은 우선순위에서 밀립니다.
- 노드 수가 n 인 균형 트리의 깊이는 $\Theta(\log n)$ [Alg 12]. max_depth=3이면 잎이 최대 $2^3 = 8$ 개.

※ BFS (level-order)는 같은 깊이의 노드를 먼저 모두 방문합니다. 트리를 그림으로 그릴 때 층별 배치가 BFS 순서입니다.

6.3 BFS/DFS가 직접 필요한 경우: 연결요소

데이터가 관계로 주어지면 그래프 탐색이 필요합니다. 예: 같은 이메일이나 전화번호를 공유하는 지원 기록을 한 사람으로 묶기 (중복 지원 제거).

```
from scipy.sparse.csgraph import connected_components
# A: 기록 i와 j가 같은 이메일/전화를 공유하면 1인 인접행렬 (희소)
n_people, label = connected_components(A, directed=False)
df["person_id"] = label
```

※ `connected_components`는 내부적으로 BFS/DFS를 반복합니다: 방문하지 않은 노드에서 탐색을 시작해 닿는 모든 노드에 같은 라벨. $\Theta(V + E)$. 직접 짜면 `collections.deque`로 BFS를 구현합니다.

※ 중첩 JSON을 표로 펴는 `pd.json_normalize`도 재귀적 DFS입니다.

Datathon에서는

이번 데이터(한 행 = 한 지원자, 관계 없음)에서는 그래프 탐색이 필요 없습니다. 다만 결정트리를 보조 분석으로 써서 “회귀가 놓친 interaction이 있는지” 찾는 데는 유용합니다. 여기서도 Engineering × 성별을 바로 짚어냈습니다.

7 Heuristic과 greedy: 최적해 대신 좋은 해

7.1 결정트리는 greedy heuristic

각 노드에서 “지금 불순도(Gini)를 가장 많이 줄이는 분할 하나”만 고르고 되돌아가지 않습니다. 전체적으로 최적인 트리를 찾는 문제는 NP-hard라서, greedy가 현실적인 대안입니다.

- 각 노드에서 변수마다 값을 정렬해 가능한 분할점을 모두 훑습니다. sklearn 소스 (`_partitioner.pyx`)에서 쓰는 정렬은 `introsort` (2장). 그래서 트리 학습 비용은 대략 $O(d \cdot n \log n)$ per level입니다.

7.2 변수 선택: forward stepwise (greedy)

변수 조합은 2^p 개라 다 볼 수 없습니다. AIC (통계책 이전 책 §7)를 평가 함수로 놓고, 매 단계 AIC를 가장 많이 줄이는 변수 하나를 추가합니다.

단계	후보별 AIC (logistic)	선택	AIC
0	(절편만)	–	735.8
1	부서 679.4, 점수 701.2, 성별 706.5, 경력 715.1, 학위 731.2	부서	679.4
2	점수 638.4, 경력 667.9, 성별 675.4, 학위 677.3	점수	638.4
3	경력 625.9, 성별 635.8, 학위 635.9	경력	625.9
4	학위 623.3, 성별 623.5	학위	623.3
5	성별 620.7	성별	620.7

해석: 성별은 마지막에 들어오지만 그래도 AIC를 낮춥니다 ($623.3 \rightarrow 620.7$). 다른 모든 자격 변수를 넣은 뒤에도 성별이 설명력을 더한다는 뜻으로, 통계책 13장의 “통제 후 성별 계수 유의”와 같은 결론입니다.

※ Greedy의 한계: 단계 1에서 성별(706.5)은 점수(701.2)와 비슷했지만 부서와 겹치는 정보라 뒤로 밀렸습니다. 순서가 결과 해석을 좌우하므로, stepwise는 탐색용이고 최종 모형은 이론(무엇을 통제해야 하는가)으로 정합니다.

7.3 Heuristic function과의 관계

- A* 탐색의 heuristic function $h(n)$: 목표까지 남은 비용의 추정치. 정확하지 않아도 탐색 방향을 잡아 줍니다. Admissible ($h \leq$ 실제 비용)이면 최적해 보장.
- 데이터 분석의 “heuristic”: Gini 감소, AIC, validation 오차 같은 평가 함수로 거대한 탐색 공간 (트리 구조, 변수 조합, 하이퍼파라미터)을 greedy하게 훑는 것.
- 하이퍼파라미터 탐색: `GridSearchCV` (전수 탐색), `RandomizedSearchCV` (무작위), Bayesian optimization (모형 기반 heuristic).

8 Minimax: 직접은 없지만 같은 생각이 있다

8.1 원래 의미

게임 트리에서 내 차례에는 max, 상대 차례에는 min을 취해 최악의 상대를 가정했을 때 가장 좋은 수를 고릅니다. 구현은 DFS + 깊이 제한 + heuristic 평가 함수 (+ alpha-beta 가지치기). 데이터 분석에 게임 트리가 나올 일은 거의 없습니다.

8.2 같은 원리가 나오는 곳

1. Minimax 추정량 (통계적 결정이론). 추정량의 위험(MSE)은 진짜 θ 에 따라 달라집니다 (통계책 9장). “ θ 가 최악일 때의 MSE를 가장 작게” 만드는 추정량이 minimax 추정량입니다:

$$\hat{\theta}_{\text{minimax}} = \arg \min_{\hat{\theta}} \max_{\theta} \text{MSE}(\hat{\theta}; \theta).$$

※ 자연(nature)을 “나를 괴롭히는 상대”로 보는 게임입니다. Bayesian은 반대로 θ 에 대한 prior로 평균 위험을 최소화합니다.

2. Minimax 공정성 (fairness). “가장 불리한 그룹의 성과를 최대화”하는 기준입니다:

$$\max_{\text{정책}} \min_{\text{그룹 } g} \text{성과}_g.$$

```
g = df.groupby(["department", "gender"]).hired.mean()
g.idxmin(), g.min() # (Marketing, M), 0.098
g.idxmax(), g.max() # (Engineering, M), 0.527
```

※ 평균 격차(demographic parity)만 보면 놓치는 것: 가장 불리한 셀은 여성이 아니라 *Marketing* 남성 (9.8%)입니다. 다만 61명뿐이라 불확실성이 큼 (통계책 11장 검정력). 발표에서 “최악 그룹”을 함께 보여 주면 분석의 깊이가 드러납니다.

3. Robust 결론. 여러 모형(LPM, logistic, Bayesian, 결정트리) 중 가장 불리한 모형에서도 결론이 유지되는가를 보는 것도 minimax식 사고입니다. 이번 데이터에서는 Engineering 성별 격차가 모든 방법에서 나왔습니다.

9 Dijkstra: 이번 datathon에는 필요 없다

9.1 언제 쓰나

가중치가 음수가 아닌 그래프에서 한 출발점으로부터 모든 노드까지의 최단 경로. 데이터 형태가 노드와 간선일 때만 필요합니다.

datathon 주제 예	쓰임
교통·물류 (버스 노선, 배송)	지역 간 최단 이동 시간, 접근성 지표
소셜 네트워크	사람 사이 거리, 중심성 (closeness)
인프라 (전력망, 상수도)	장애 시 우회 경로

```
from scipy.sparse.csgraph import dijkstra          # 또는 networkx.shortest_path_length
dist = dijkstra(adjacency_matrix, indices=source)
```

9.2 알고리즘의 연결고리

- 핵심 자료구조: **min-heap** 우선순위 큐. 매번 “지금까지 거리가 가장 짧은 미방문 노드”를 꺼냅니다 (ExtractMin, $O(\log V)$). 강의 13의 힙 연산 그대로이고, 3장의 top- k , 6장의 best-first tree builder 와 같은 도구입니다.
- 복잡도: 이진 힙으로 $O((V + E) \log V)$.
- 가중치가 모두 같으면 Dijkstra = **BFS** (6장). A*는 Dijkstra에 heuristic function h 를 더한 것 (7장).

※ 이번 채용 데이터는 지원자 사이의 관계가 없는 표 데이터라 그래프가 만들어지지 않습니다. 그러니 이번 datathon에서는 넘겨도 됩니다.

10 분할정복과 점화식: 분석 파이프라인의 비용

[Alg 10 Merge sort, 11 Max subarray, 18–19 Recurrences, 02 Peak finding]

10.1 Split–apply–combine

`groupby().apply(f)`는 데이터를 그룹으로 나누고 (split), 각각 계산하고 (apply), 합칩니다 (combine). 분할정복의 divide–conquer–combine과 같은 구조입니다. 차이: 재귀가 아니라 한 단계라서 비용은 $\Theta(n) + \sum_g \text{cost}(f, n_g)$.

10.2 재표본 방법의 비용

방법 (통계책)	비용	이번 데이터
Bootstrap B 회 (9장)	$\Theta(B \cdot n) \times$ 통계량 비용	2000×600 , 수 초
Permutation P 회 (12장 ⑬)	$\Theta(P \cdot n)$	5000×600
k -fold cross-validation	$k \times$ 모형 학습 비용	–
결정트리 학습	대략 $O(dn \log n \cdot \text{depth})$	순식간

※ B 를 2배로 하면 시간도 2배, Monte Carlo 오차는 $1/\sqrt{2}$ 배. “정밀도를 2배로 하려면 4배의 반복”이라는 $\sqrt{n}$ 법칙이 시뮬레이션 비용에도 적용됩니다.

10.3 강의의 다른 알고리즘이 쓰이는 곳

- **Maximum subarray** [Alg 11]: 시계열에서 “누적 이득이 가장 큰 구간” (예: 주가가 가장 많이 오른 기간, 합격률이 가장 높았던 연속 기간). Kadane 알고리즘 $\Theta(n)$. pandas로는 `cumsum`과 `cummin`의 조합으로 벡터화할 수 있습니다.
- **Peak finding** [Alg 02]: 시계열의 국소 최댓값. `scipy.signal.find_peaks`.
- **Binary search on answer**: “합격률 차이가 0.8 비율(four-fifths)을 넘는 최소 점수 기준”처럼 단조인 조건의 경계 찾기.

※ 이번 채용 데이터에는 시간 축이 없어서 이 두 알고리즘은 쓰이지 않습니다.

11 정리: datathon 실전 규칙

1. 행 반복 금지. `iterrows`, `apply(axis=1)` 대신 열 연산 (1장, 110–230배).
2. 이중 루프 금지. 두 표의 짝짓기는 `merge` (4장, 1400배, 차수의 차이).
3. 정렬은 기본값으로 충분. 여러 키 정렬은 `sort_values([k1, k2])` 한 번에. 단계별로 하면 `kind="stable"` (2장).
4. 상위 k 개는 `nlargest`, 분위수는 `quantile` (3장).
5. 구간 나누기는 `pd.cut/qcut`, 정렬된 자료 탐색은 `searchsorted` (5장). 전제조건: 정렬.
6. 결정트리는 `interaction` 탐색용 보조 도구, 결론은 회귀와 검정으로 (6장).
7. Stepwise 선택은 탐색용, 최종 모형은 이론으로 (7장).
8. 최악 그룹도 보고: `groupby(...).mean().idxmin()` (8장).
9. 그래프 알고리즘 (BFS/DFS/Dijkstra)은 데이터가 관계일 때만 (6, 9장).

12 연습 문제

1. `df.sort_values("test_score")`의 기본 알고리즘은 무엇이고, 최악의 경우 시간복잡도는? 순수 quick sort와 무엇이 다른가?
2. 100만 개의 `years_experience` (0-11 정수)를 정렬할 때 `kind="stable"`이 quicksort보다 훨씬 빠를 수 있는 이유를 정렬 하한과 연결해 설명하시오.
3. 부서 오름차순, 같은 부서에서는 점수 내림차순으로 정렬하려고 다음과 같이 썼다.

```
d.sort_values("department").sort_values("test_score", ascending=False,
kind="stable")
```

무엇이 틀렸는가?

4. 중앙값을 quickselect로 구하는 평균 시간이 $\Theta(n)$ 인 이유를 점화식으로 보이시오.
5. 크기 k 인 min-heap으로 n 개 중 상위 k 개를 구하는 알고리즘을 쓰고, 복잡도가 $O(n \log k)$ 임을 보이시오. 왜 max-heap이 아니라 min-heap인가?
6. 지원자 표 (10만 행)와 면접 표 (10만 행)를 지원자 ID로 이중 for문으로 합치면 4장 측정(3000행에 0.45초)을 기준으로 대략 얼마나 걸리겠는가? merge로는?
7. 결정트리에서 `max_depth=3`과 `max_leaf_nodes=4`는 각각 어떤 탐색 방식(자료구조)으로 트리를 키우는가?
8. Forward stepwise에서 성별이 마지막에 선택되었다. 이것이 “성별은 가장 덜 중요하다”는 뜻인가?

13 풀이

1.

Answer. Introsort, 최악 $O(n \log n)$. 순수 quick sort는 최악 $\Theta(n^2)$ (매번 최솟값/최댓값이 pivot)이지만, introsort는 재귀 깊이가 $c \log n$ 을 넘으면 heap sort로 전환해 최악을 막습니다.

2.

Answer. 비교 정렬의 하한 $\Omega(n \log n)$ 은 원소를 비교로만 다룰 때의 하한입니다. 값의 범위가 작은 정수($k = 12$)면 counting/radix sort로 개수를 세서 $\Theta(n + k) = \Theta(n)$ 에 정렬할 수 있습니다. numpy의 `stable`은 작은 정수형에 radix sort를 씁니다 (2장 측정: 27배).

Justification. 단, `years_experience`가 `int64`로 저장되어 있으면 radix 대신 timsort가 쓰일 수 있습니다. `astype("int8")`로 줄이면 이득을 봅니다.

3.

Answer. 순서가 거꾸로입니다. `stable` 다단계 정렬은 덜 중요한 키를 먼저, 더 중요한 키를 나중에 `stable`하게 정렬해야 합니다. 올바른 코드:

```
d.sort_values("test_score", ascending=False).sort_values("department", kind="stable")
d.sort_values(["department", "test_score"], ascending=[True, False])      # 권장
```

Justification. Radix sort가 낮은 자리부터 정렬하는 이유와 같습니다 [Alg 17].

4.

Answer. 평균적으로 partition이 배열을 비슷하게 나누면 한쪽만 재귀하므로 $T(n) = T(n/2) + cn$. 펼치면 $cn(1 + \frac{1}{2} + \frac{1}{4} + \dots) \leq 2cn = \Theta(n)$.

Justification. Quick sort는 양쪽 모두 재귀해서 $T(n) = 2T(n/2) + cn = \Theta(n \log n)$. 차이는 재귀 호출이 1개냐 2개냐입니다.

5.

Answer. 처음 k 개로 min-heap을 만들고 ($O(k)$), 나머지 각 원소 x 에 대해 $x >$ 루트이면 루트를 x 로 바꾸고 HeapifyDown ($O(\log k)$). 총 $O(k + (n - k) \log k) = O(n \log k)$.

Justification. 힙의 루트는 “지금까지 상위 k 개 중 가장 작은 값”, 즉 탈락 후보입니다. 새 원소와 비교할 대상이 이것이므로 min-heap이어야 루트에서 $O(1)$ 로 확인할 수 있습니다.

6.

Answer. 이중 루프는 $\Theta(nm)$ 이므로 $(10^5/3000)^2 \approx 1111$ 배 $\Rightarrow$ 약 $0.45 \times 1111 \approx 500$ 초 (8분 이상). `merge`는 $\Theta(n + m)$ 이므로 약 33배 $\Rightarrow$ 약 0.01초.

7.

Answer. `max_depth:` DepthFirstTreeBuilder, 스택 기반 DFS. `max_leaf_nodes:` BestFirstTreeBuilder, 불순도 감소량을 키로 하는 우선순위 큐 (힙).

8.

Answer. 아닙니다. Greedy는 각 단계에서 그 시점의 추가 이득만 봅니다. 성별의 정보 일부가 부서와 겹치므로 (성별 $\times$ 부서 $\chi^2 = 132$), 부서가 먼저 들어가면 성별의 추가 이득이 작아 보입니다. 그래도 마지막에 AIC를 낮췄다는 것은 모든 자격 변수를 통제 한 후에도 성별이 설명력을 가진다는 뜻이고, 이것이 bias 분석의 핵심 질문입니다.